CASE STUDY

Netflix Scaling & High-Availability Architecture

Course: Web Application Architecture

Focus: Real-world DevOps and Scalability

Prepared by: [Shariqua Patel Farooque]

Student ID: [DVP06018]

Date: March 5, 2026

Table of Contents

1. Introduction & Architecture Type ..... [Page 4]

- **1.1 Microservices vs. Monolithic Systems**
- **1.2 The Role of Amazon Web Services (AWS)**

2. Tier 1: Client Delivery & The Edge ..... [Page 5]

- **2.1 Open Connect (CDN): Global Edge Caching**
- **2.2 Load Balancing with Amazon ELB**
- **2.3 Auto-scaling and EC2 Virtual Servers**
- **2.4 The API Gateway: Netty Server and Zuul**

3. Tier 2: The Microservices & Resilience Layer ..... [Page 6]

- **3.1 Decoupled Service Functions**
- **3.2 Fault Tolerance with Hystrix (Circuit Breaker)**
- **3.3 Chaos Engineering: The Chaos Monkey**
- **3.4 Container Management via TITUS**

4. Tier 3: Data Storage & Caching ..... [Page 7]

- **4.1 Polyglot Persistence Strategy**
- **4.2 High-Speed Caching with EVCache**
- **4.3 NoSQL vs. Relational: Cassandra and MySQL**

5. Tier 4: Video Processing & Transcoding Pipeline ..... [Page 8]

- **5.1 Queues and Concurrent Async Workers**
- **5.2 Storage and CDN Push via Amazon S3**

6. Tier 5: Event Processing & Analytics ..... [Page 9]

- **6.1 Real-time Collection: Chukwa and Kafka**
- **6.2 Stream Processing: Apache Samza and Spark**
- **6.3 Historical Data & Batch Processing (Amazon EMR)**

7. Design Goals: The Engineering Philosophy ..... [Page 10]

- **7.1 High Availability and Fault Isolation**
- **7.2 Resilience and Personalization at Scale**
- **7.3 Data Integrity for Critical Flows**

8. Netflix Architectural Summary Table ..... [Page 11]

9. Conclusion ..... [Page 12]

10. References ..... [Page 12]

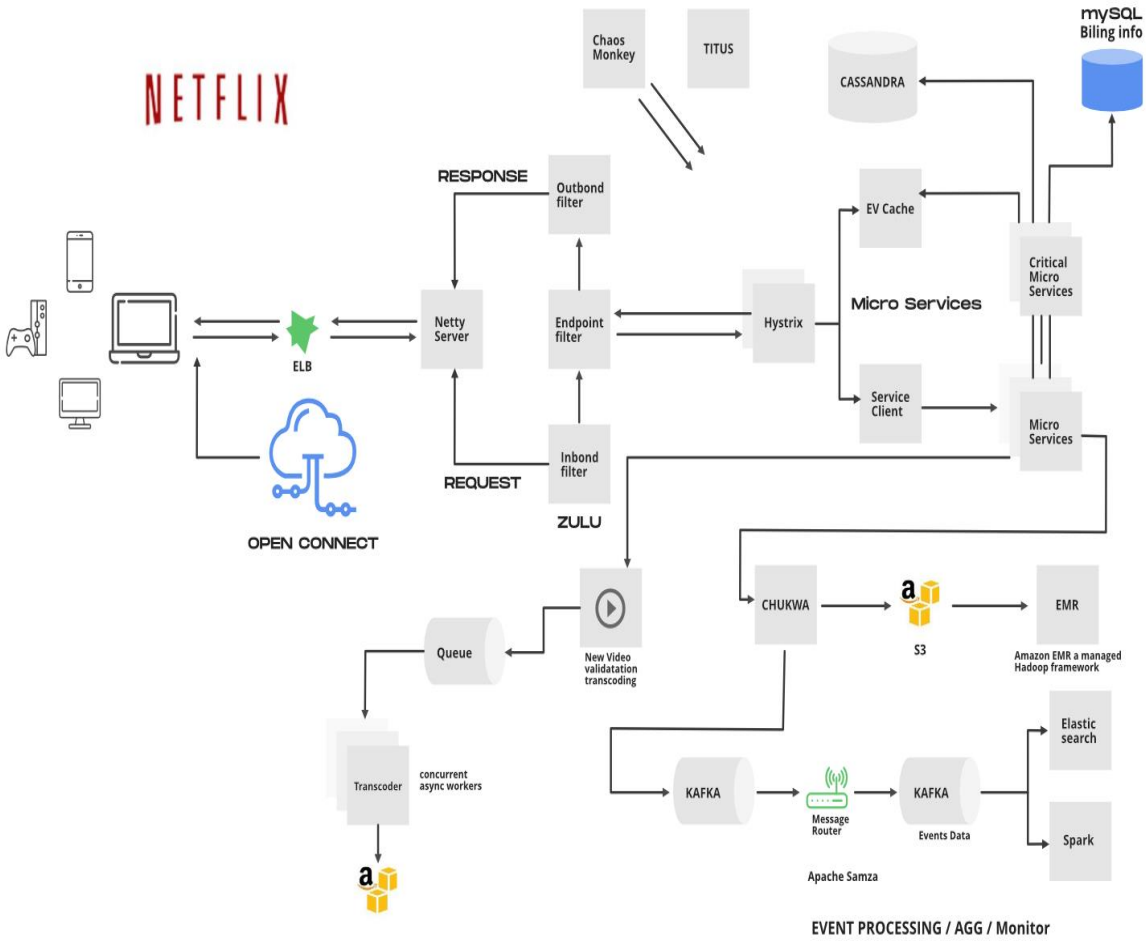
Case Study: Netflix Scaling & High-Availability Architecture

Course: Web Application Architecture

Focus: Real-world DevOps and Scalability

1. Introduction & Architecture Type

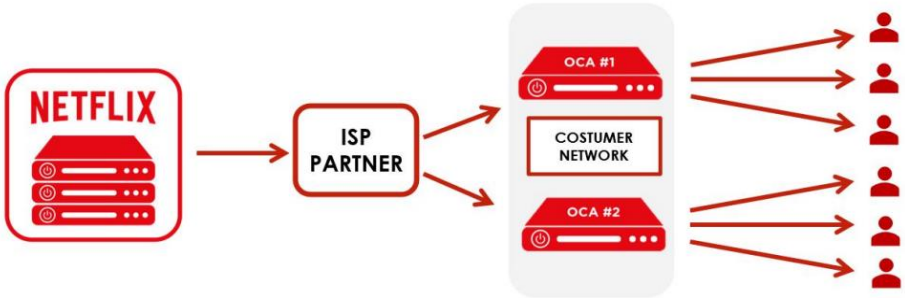
Netflix is the world's leading streaming service, serving over 200 million users globally. To manage this massive scale, Netflix uses a **Microservices Architecture** hosted primarily on **Amazon Web Services (AWS)**. Unlike a monolithic system where everything is one giant program, Netflix is split into over 1,000 independent services that communicate via APIs.



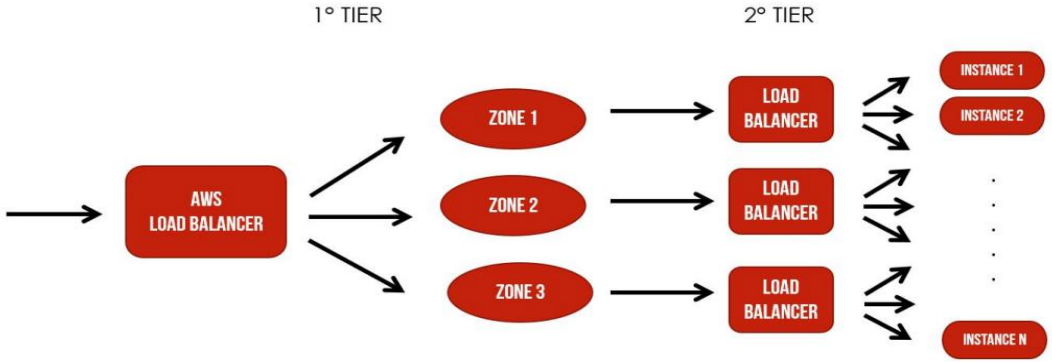
2. Tier 1: Client Delivery & The Edge

This tier is responsible for how the user interacts with the app and how data enters the Netflix system.

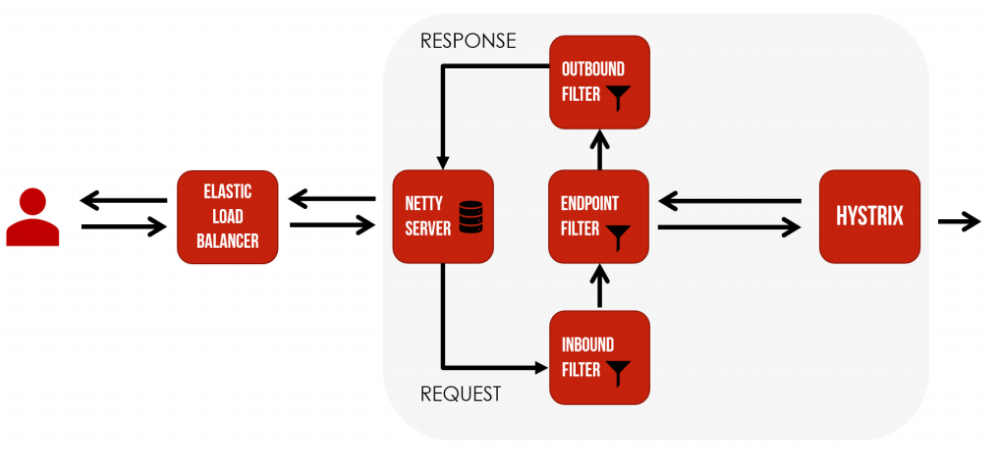
- **Open Connect (CDN):** Netflix's proprietary Content Delivery Network (CDN). It caches video files at the "edge"—physically close to the user—to ensure high-speed, 4K streaming without buffering.



- **Load Balancing (ELB):** Uses Amazon's Elastic Load Balancer to distribute traffic evenly across servers so no single point fails under pressure.
- **Scaling:** They use **Auto-scaling** to spin up thousands of new servers during "Prime Time" (evenings) and shut them down late at night to save costs.
- Amazon **EC2 (Elastic Compute Cloud)** is a web service that provides **resizable virtual servers** in the cloud. In this architecture, EC2 acts as the physical-like host where our software (Zuul, Netty, and Microservices) lives and breathes. Instead of buying physical hardware, we "rent" these virtual machines by the second.



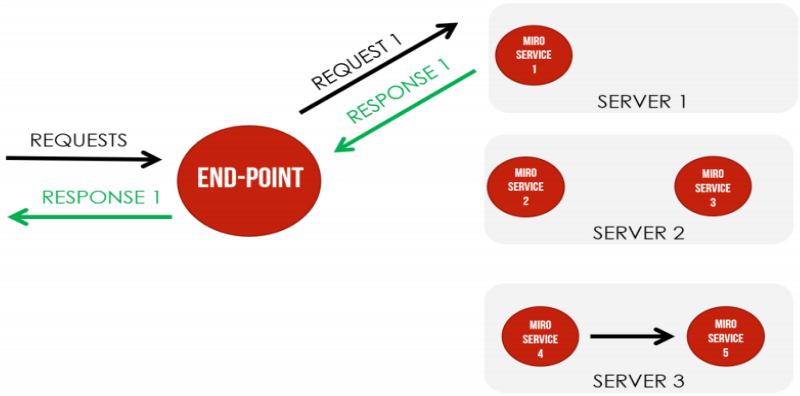
- **Web Server (Netty):** Incoming requests are handled by the **Netty Server**, an asynchronous network framework that can handle thousands of concurrent user connections with low memory usage.



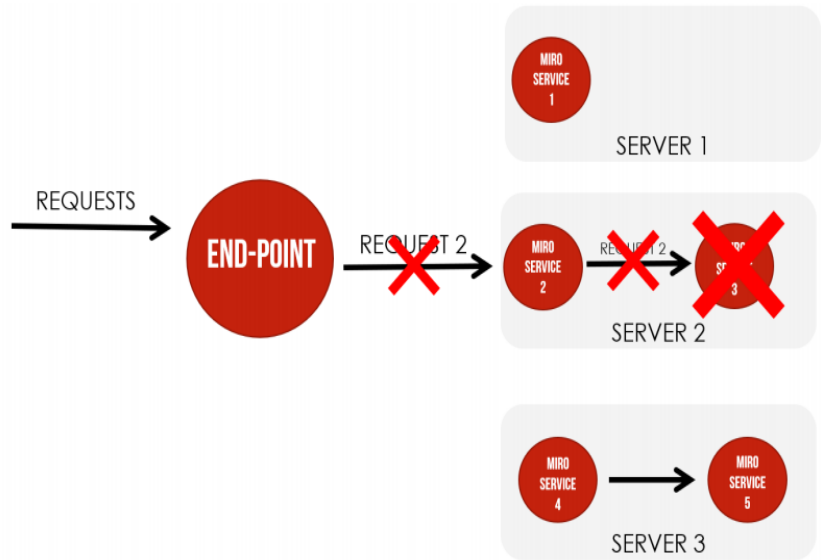
- **API Gateway (Zuul):** Labeled as "ZULU" in the diagram, this acts as the "front door". It uses **Inbound Filters** for security, **Endpoint Filters** for routing, and **Outbound Filters** to package the response back to the user.

3. Tier 2: The Microservices & Resilience Layer

The "brain" of Netflix consists of decoupled services that handle specific tasks like searching, movie recommendations, or user profiles.



- **Hystrix (Circuit Breaker):** This sits between microservices to prevent cascading failures. If one service is slow, Hystrix "trips the circuit" to protect the rest of the system.

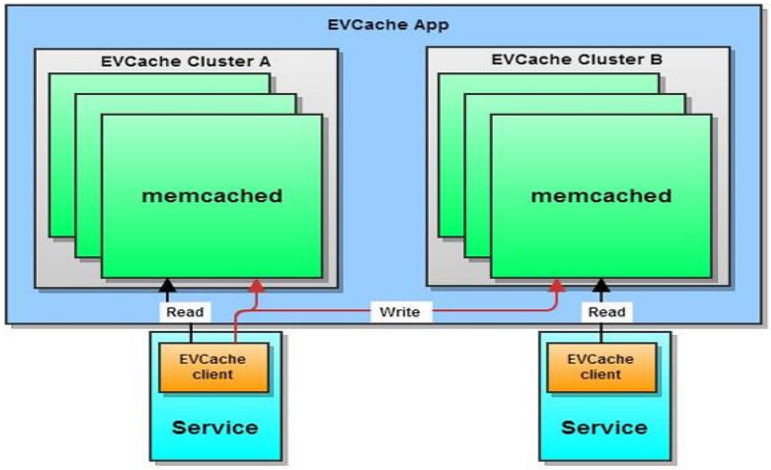


- **Chaos Monkey:** A unique tool that intentionally kills production servers to test if the system can automatically recover.
- **TITUS:** Netflix's container management platform (similar to Kubernetes) that manages how these microservices are deployed and run.

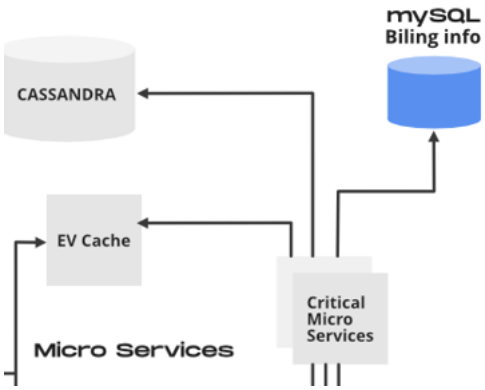
4. Tier 3: Data Storage & Caching

Netflix uses "Polyglot Persistence," meaning they use different databases for different jobs.

- **EVCache:** A high-speed, in-memory cache used for frequently accessed data (like your "Watch List") to reduce load on the main databases.



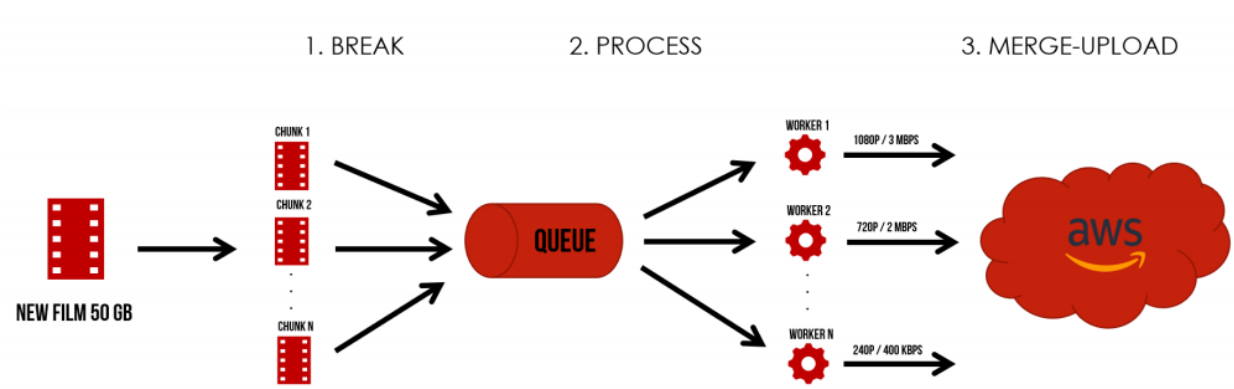
- **Cassandra:** A NoSQL database used for massive amounts of user data that requires high availability across different global regions.



- **MySQL (Billing Info):** Used specifically for billing because it requires strict transactional accuracy (ACID compliance).

5. Tier 4: Video Processing & Transcoding Pipeline

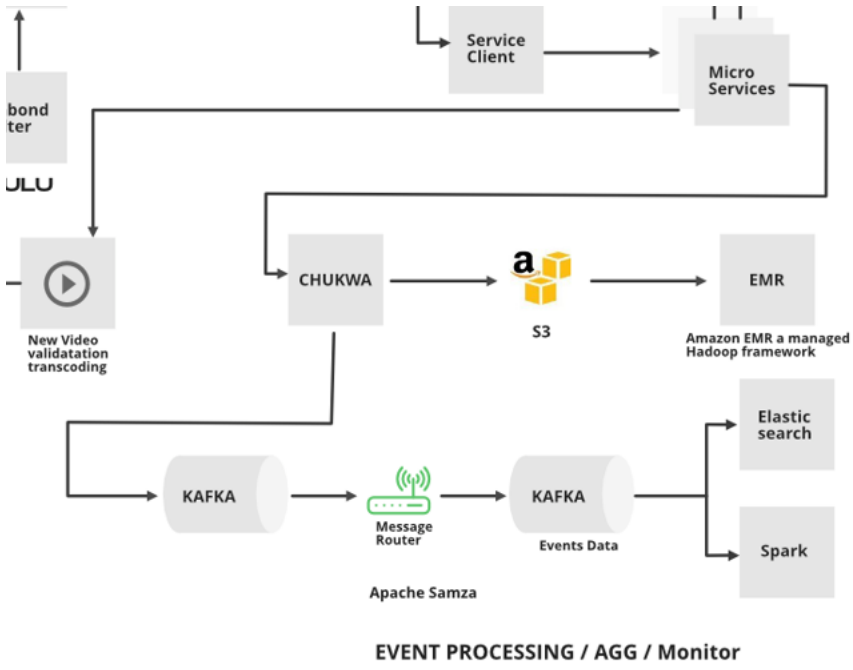
When a new movie is added to Netflix, it must be prepared for every possible device.



- **Queues & Transcoders:** New videos are placed in a **Queue** where **Concurrent Async Workers (Transcoders)** break the video down into thousands of small chunks.
- **Amazon S3:** The processed video chunks are stored in **S3 buckets** before being pushed to the Open Connect CDN for users to watch.

6. Tier 5: Event Processing & Analytics

This tier monitors everything that happens on the platform to improve recommendations and detect errors.



- **Data Collection (Chukwa & Kafka): Chukwa** collects logs while **Kafka** handles massive streams of real-time event data (e.g., every time you click "Play").
- **Real-time Processing (Apache Samza & Spark): Samza** routes messages, and **Spark** performs high-speed analytics for real-time insights.
- **Batch Processing (Amazon EMR):** Uses a managed Hadoop framework to process massive amounts of historical data stored in S3 for long-term business reports.
- **Elasticsearch:** Used to index logs and data so engineers can quickly search for errors or system issues.

7. Design Goals: The Engineering Philosophy

Netflix engineers their system around five core "North Star" goals:

- **High Availability (99.99% Uptime):** The system is designed so that the "Play" button always works. They achieve this through **Fault Isolation**—ensuring a failure in the "Rating" service never stops the "Streaming" service.
- **Low Latency & Scalability:** The system must start a video in under 2 seconds for 200 million people simultaneously. This is achieved via **Open Connect CDNs** and **Auto-scaling EC2 instances**.
- **Resilience by Design:** Instead of trying to prevent every failure, Netflix assumes failure is inevitable. They use **Chaos Engineering** to test this daily.
- **Personalization at Scale:** The goal is to provide a unique "Homepage" for every user. This requires processing **35 billion events daily** using Kafka and Spark to update recommendations in real-time.
- **Data Integrity for Critical Flows:** While viewing history can be "eventually consistent," **Billing and Subscription data** must be 100% accurate. This is why they use **MySQL (RDS)** for financial transactions.

8. Netflix Architectural Summary Table

Category	Technology / Framework	AWS Service Used	Primary Role
Edge & Gateway	Netty & Zuul	ELB & Route 53	Manages traffic routing, security, and load balancing.
Compute	Java, Python, Node.js	EC2 & Lambda	Powers microservices, backend logic, and serverless tasks.
Architecture	Microservices	Titus (on EC2)	Decouples services for independent scaling and deployment.
Primary Storage	Video Assets	Amazon S3	Stores raw and transcoded video files (Source of Truth).
NoSQL Database	Cassandra / DynamoDB	EC2 / DynamoDB	Stores massive user metadata and viewing history.
Relational DB	MySQL / PostgreSQL	Amazon RDS	Ensures ACID compliance for Billing and Finance.
Caching	EVCache	EC2 / ElastiCache	Provides sub-millisecond data retrieval for "My List."
Big Data	Kafka & Spark	Kinesis & EMR	Processes real-time events and recommendation analytics.
Monitoring	Chukwa & Elasticsearch	S3 & EC2	Collects logs and provides searchable system telemetry.
Resilience	Hystrix & Chaos Monkey	EC2	Implements fault tolerance and "Chaos Engineering."
Content Delivery	Open Connect (CDN)	Custom Hardware	Serves video bits from local ISPs to prevent buffering.

9. Conclusion

Netflix's transition from a DVD-by-mail service to a global streaming giant is a result of its "Cloud-First" and "Microservices" strategy. By leveraging AWS for its massive compute needs and building its own Open Connect CDN for video delivery, Netflix successfully balanced innovation with reliability.

The most significant takeaway from this case study is that scalability is as much about culture as it is about code. Through principles like "Context not Control" and tools like "Chaos Monkey," Netflix has built a self-healing system that defines the modern standard for DevOps and distributed systems.

10. References

1. Tech Dummies - Narendra Lakshmana Gowda. (2018). *NETFLIX System Design | Software Architecture for Netflix.* [YouTube Video]. Available at: https://youtu.be/psQzyFfsUGU

2. Netflix Technology Blog. (2025). *Behind the Streams: Building a Reliable Cloud Live Streaming Pipeline.* https://netflixtechblog.com

3. Amazon Web Services (AWS). *Case Study: Netflix on AWS.* https://aws.amazon.com/solutions/case-studies/netflix/

4. Netflix Research. *Personalization and Recommender Systems.* https://research.netflix.com/

5. GeeksforGeeks. (2025). *System Design Netflix | A Complete Architecture.* https://www.geeksforgeeks.org/system-design-netflix-a-complete-architecture/